Fast Fourier Transform (FFT) in Competitive Programming

From $O(N^2)$ Math to $O(N \log N)$ Code

Moving beyond basic algorithms to solve complex convolution and string matching problems — a deep dive into the mathematics and implementation that turns impossible constraints into accepted solutions.

ADVANCED ALGORITHMS | COMPETITIVE PROGRAMMING | COMPLEX NUMBERS

Made with GAMMA

The Core Problem: Polynomial Multiplication

The Bottleneck

A polynomial $P(x) = a_0 + a_1x + a_2x^2 + \ldots$ is stored as a coefficient vector $[a_0, a_1, a_2, \ldots]$. Naive multiplication of two degree-N polynomials requires computing every pair product — $\mathbf{O(N^2)}$ operations. For $N = 10^5$, this means 10^{10} ops and guaranteed **TLE**.

The Solution: Point-Value Representation

A degree-(N-1) polynomial is uniquely determined by its values at N distinct points. If we know $P(x_i)$ and $Q(x_i)$ at N points, then:

> $\mathbf{(P \cdot Q)(x_i) = P(x_i) \times Q(x_i)}$ — computed in O(N) time!

The challenge: convert between coefficient and point-value forms *efficiently*. FFT does this in O(N log N).

Made with GAMMA

The Magic Points: N-th Complex Roots of Unity

We evaluate at the **N-th roots of unity** — complex numbers satisfying $\omega^n = 1$. The primitive root is:

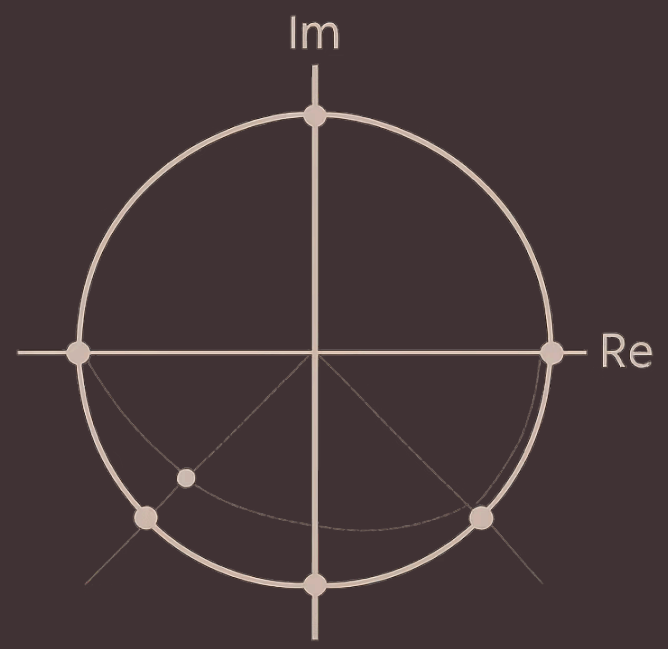
$$\omega_n = e^{2\pi i/n} = \cos\frac{2\pi}{n} + i\sin\frac{2\pi}{n}$$

Squaring Property

$(\omega_n^k)^2 = \omega_{n/2}^k$ — halves the problem size each recursion level.

Symmetry Property

$\omega_n^{k+n/2} = -\omega_n^k$ — evaluations at ω and $-\omega$ share work.

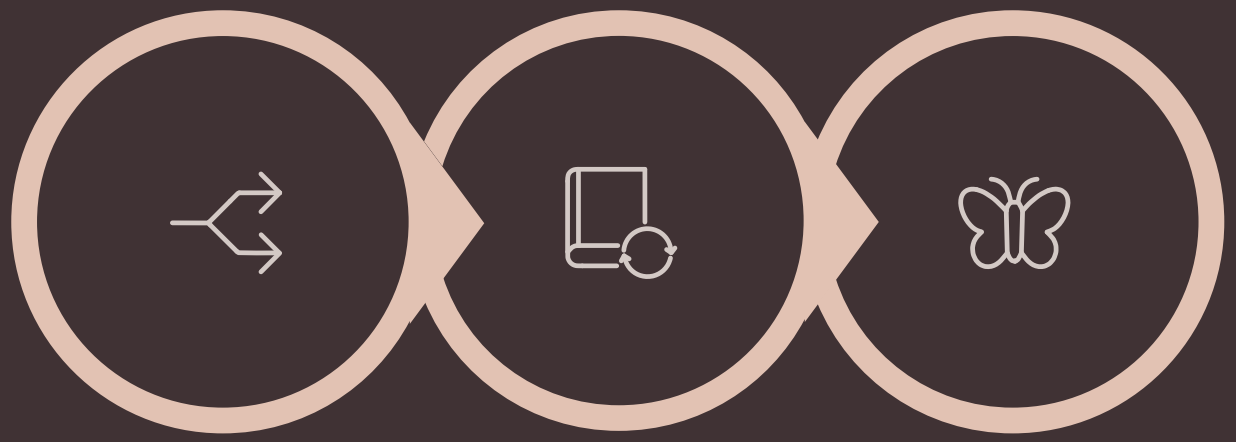


Made with GAMMA

The Math: Divide & Conquer

The "Fast" in FFT comes from splitting P(x) into even and odd indexed coefficients:

$$P(x) = P_{\text{even}}(x^2) + x \cdot P_{\text{odd}}(x^2)$$



Split **Recurse** **Combine**

Evaluating at ω_n^k and $\omega_n^{k+n/2} = -\omega_n^k$ reuses the same sub-computation. The recurrence $T(N) = 2T(N/2) + O(N)$ solves to **O(N log N)** by the Master Theorem.

Made with GAMMA

The Butterfly Operation

The Core Computation

Let $t = \omega_n^j \cdot o[j]$ be computed **once**. Then both outputs are updated simultaneously:

$$c[j] = e[j] + t$$

$$c[j + m/2] = e[j] - t$$

This "butterfly" pattern is the atomic unit of FFT — one complex multiply updates two array positions.

Why It Matters

Halves Multiplies

One complex multiply serves two outputs — critical for constant-factor optimization.

In-Place

Butterfly operations can be performed in-place, reducing memory to O(N).

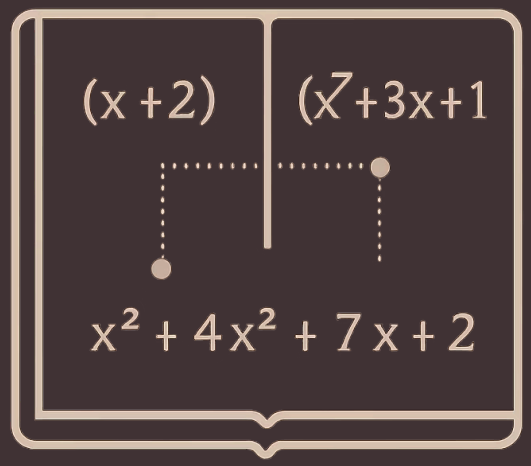
Cache-Friendly

Iterative bottom-up ordering improves cache locality over recursive versions.

Made with GAMMA

PROBLEM 1 — SANITY CHECK

SPOJ — POLYMUL



Problem Statement

Multiply two polynomials of degree up to 10^5. Input: coefficient arrays A and B. Output: coefficients of A × B.

Why Start Here

POLYMUL is the canonical FFT correctness test. If your template fails here, the bug is in your core FFT — not the problem modeling.

⚠ **Debug checklist:** Verify bit-reversal produces correct permutation, confirm butterfly uses the right root power, check output normalization (divide by N after inverse FFT).

Made with GAMMA

PROBLEM 2 — HIDDEN CONVOLUTION

CSES 2111 — Apples and Bananas

Problem

Given apple weights and banana weights, count the number of ways to form every possible combined weight by picking one apple and one banana.

Convolution Insight

Let A[x] = count of apples with weight x, B[x] = count of bananas with weight x. Then:

$$[x^k]\,(A(x) \cdot B(x)) = \sum_i A[i] \cdot B[k-i]$$

This is exactly polynomial multiplication — the coefficient of x^k is the answer for weight k.

Modeling Steps

→ Frequency Arrays

Map each weight to its frequency. Array index = weight, value = count.

→ FFT Multiply

Run NTT on both frequency arrays, point-wise multiply, inverse NTT.

→ Read Answers

Result[k] gives the number of (apple, banana) pairs summing to weight k.

Made with GAMMA

PROBLEM 3 — PATTERN MATCHING

Codeforces 954I — Yet Another String Matching Problem

The Challenge

For each cyclic shift of string B, find the minimum character replacements needed to make it equal to A. Naive $O(N^2)$ per shift is too slow.

FFT Reduction

For each character c ∈ {'a'–'z'}, build boolean arrays: $P_c[i] = 1$ if A[i] = c, and $Q_c[j] = 1$ if B[j] = c. Reverse Q_c so convolution becomes shift matching:

$$(P_c * Q_c^R)[N-1-k] = \text{matches of char } c \text{ at shift } k$$

DSU Integration

For each shift, FFT gives match counts per character. The character with maximum matches requires minimum replacements. Use **Disjoint Set Union** to track which positions can be made equal with minimum operations across all character classes.

Key trick: Reversing Q converts $x + y = k$ (standard convolution) into $x - y = k$ (shift alignment).

Made with GAMMA

SUMMARY

Resources & Problems

Resources:

Codeforces

[Tutorial] FFT – Codeforces

Codeforces. Programming competitions and contests, programming community

cp-algorithms.com

Fast Fourier transform – Algorithms for Competitive Progra...

The goal of this project is to translate the wonderful resource http://e-maxx.ru/algo which provides descriptions of many algorithms and...

Problems:

Codeforces

Problem – G – Codeforces

Codeforces. Programming competitions and contests, programming community

Codeforces

Problem – E – Codeforces

Codeforces. Programming competitions and contests, programming community

The CP Mindset for FFT

FFT is a Black Box

Once your template is tested, treat it as a library call. Focus mental energy on problem modeling.

Spot the Convolution

The real skill is recognizing $C_k = \sum A_i \cdot B_{k-i}$ hidden in combinatorics, strings, or geometry.

Pre-Write Your Template

Have a tested FFT/NTT template ready before every contest. Debugging it under time pressure is a losing strategy.

Made with GAMMA